B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institute, Affiliated to VTU)

Course – Mobile Application Development
Course Code – 22IS5PCMAD

Topic: StudyBuddy: Your Personal Study And Sanity Assistant

Submitted By
Disha D (1BM22IS250)
Rida Rabeea Fatima (1BM22IS155)
Shama (1BM22IS179)

Under the guidance of
Rashmi R
Assistant Professor

DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

AY 2025-2026

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institute, Affiliated to VTU)
Bull Temple Road, Basavanagudi,
Bengaluru – 560019

Department of Information Science and Engineering

C E R T I F I C A T E

This is to certify that the project entitled "**StudyBuddy: Your Personal Study And Sanity Assistant**" is a bona-fide work carried out by Disha D(1BM22IS250), Rida Rabeea Fatima(1BM22IS155), Shama(1BM22IS179) in partial fulfilment for the award of degree of Bachelor of Engineering in **Information Science and Engineering** from **Visvesvaraya Technological University, Belgaum** during the year **2024-2025**. It is certified that all corrections/suggestions indicated for Internal Assessments have been incorporated in the report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of the project work prescribed for the Bachelor of Engineering Degree.

Rashmi R	**Dr. Nalini MK**	**Dr.Bheemsha Arya**
Assistant Professor	**Associate Professor and HOD**	**Principal**

Examiners

Name of the Examiner	**Signature of the Examiner**
1.	
2	

DECLARATION

NAME (USN) students of B.E. Information Science and Engineering, B.M.S. College of Engineering, Bangalore - 19, hereby declare that the capstone project entitled **"StudyBuddy: Your Personal Study And Sanity Assistant"** is an authentic work carried out under the supervision and guidance of **Rashmi R**, **Assistant Professor**, Department of Information Science and Engineering, B.M.S. College of Engineering, Bangalore. We have not submitted the matter embodied to any other university or institution for the award of any other degree.

Place:

Date:

Name	USN	Signature
Disha D	1BM22IS250	……
Rida Rabeea Fatima	1BM22IS155	…….
Shama	1BM22IS179	……..

TABLE OF CONTENTS

ABSTRACT:

StudyBuddy: Your Personal Study & Sanity Assistant is a holistic Android app designed to assist students both academically and emotionally. It integrates productivity resources with emotional wellness tools in a hassle-free and intuitive interface. Key features are a Smart Scheduler to organize tasks and deadlines, an Instant Doubt Solver with GPT API for instant academic assistance, and an Emotional Vent Space that replies with AI-composed soothing or motivational statements.

The app also includes Focus Mode with a timer countdown and lo-fi playlist recommendations, a Progress Tracker to see goals and hours studied, and regular Motivational Nudges to remind students during peak-stress periods. Developed with Android Studio Dolphin (Kotlin), Firebase for authentication and cloud storage, and optional integration of GPT, StudyBuddy aims to fill the gap between grades and emotional strength — providing a full-fledged companion to student life.

By combining smart study assistance and emotional support in one mobile app, StudyBuddy enables students to remain organized, inspired, and emotionally sound. It meets the increasing demand for digital tools that not only improve academic productivity but also promote mental wellness in a challenging academic scenario.

INTRODUCTION:

In contemporary learning environments, learners tend to remain under constant stress as they juggle academic demands, co-curricular activities, and physical health. Raised expectations, combined with reduced time and mental exhaustion, may be a factor in augmented tension, burnout, and deterioration in performance and emotional well-being. Whereas productivity software to handle assignments and standalone apps for mental well-being do exist, students do not get a single tool that caters to both ends with an integrated and student-oriented approach.

StudyBuddy: Your Personal Study & Sanity Assistant is built as an all-inclusive Android app with key academic assistance features integrated into tools for emotional care. It is not only meant to assist students in staying organized but also emotionally support them during their studies. The app offers a Smart Scheduler for task and deadline management, an Instant Doubt Solver based on GPT for fast academic help, and an Emotional Vent Space that provides reassuring AI-generated answers to students who need an open space for stress or anxiety venting.

Aside from these fundamental features, StudyBuddy also comes with Focus Mode — a timer and lo-fi playlist suggestion hybrid — to support deep work sessions. Developed with Android Studio Dolphin and Kotlin, Firebase for authentication and cloud storage, and optional GPT API integration, StudyBuddy uses cutting-edge development tools to provide a responsive and scalable user experience. In bringing productivity and wellness tools into one accessible system, StudyBuddy hopes to be more than an app — it hopes to be a trusted companion for students as they face the obstacles of higher education.

REQUIREMENT SPECIFICATION:

Functional Requirements

- **User Authentication**
    - Users can register and log in using Firebase Authentication.
    - User data is securely stored using Firebase Cloud Firestore.
- **Smart Scheduler**
    - Users can add, edit, and delete study tasks and deadlines.
    - The scheduler organizes tasks based on deadlines and priorities.
- **Instant Doubt Solver**
    - Users can type in academic questions or doubts.
    - The app sends queries to the GPT API and shows AI-generated answers.
- **Emotional Vent Space**
    - Users can write about how they feel or vent.
    - The app replies with AI-generated motivational or soothing messages.
- **Focus Mode**
    - Users can set a countdown timer for focused study sessions.
    - The app recommends and plays lo-fi music during sessions.
- **Motivational Nudges**
    - Sends motivational notifications at regular intervals.
    - Especially triggers nudges during peak-stress times like exams.

Non-Functional Requirements

- **Performance**
    - The app should respond quickly, especially for GPT API requests and UI actions.
- **Usability**
    - The interface should be clean, intuitive, and easy for students to use.
- **Reliability**
    - Firebase should ensure persistent storage of user data and progress.
- **Security**
    - The app should provide secure login and ensure privacy of emotional data.
- **Platform**
    - The app is developed for Android using Kotlin in Android Studio Dolphin.

SYSTEM ARCHITECTURE:

The architecture of the Student Study Sanity Assistant app is designed to be modular, scalable, and responsive to the needs of students. It integrates several cloud-based and local services to ensure a seamless experience across all features. The system architecture is primarily based on a client-server model with Firebase and OpenAI GPT APIs providing the backend services. Below is a breakdown of the core components:

1. Client-Side (Android App – Kotlin)

- Developed using Android Studio Dolphin.
- Implements Firebase Authentication SDK for user sign-up and sign-in.
- Uses Jetpack Compose/Android Views for UI rendering.
- Communicates with Firebase Firestore and external APIs using HTTPS.
- Provides intuitive navigation between modules like Scheduler, Doubt Solver, Focus Mode, Emotional Vent, and Progress Tracker.

2. Backend Services

Firebase Services

- Firebase Authentication: Manages secure user login and registration using email/password and OAuth providers if needed.
- Firebase Firestore (NoSQL DB): Stores structured user data including tasks, study logs, emotional vent entries, and focus session data.
- Firebase Cloud Messaging (FCM): Sends motivational nudges and reminder notifications.
- Firebase Storage (optional): If emotional venting involves images/audio logs in future enhancements.

OpenAI GPT API Integration

- The app sends academic queries and emotional inputs to OpenAI's GPT API.
- AI-generated responses are fetched asynchronously and displayed in-app.
- Query handling includes context formatting and basic input validation.

3. External Integrations

- Lo-fi Music API or Local Asset Player: Used to stream or play relaxing background music during Focus Mode.
- Charts Library: Utilized for progress visualization (e.g., MPAndroidChart).

4. Data Flow Overview

1. User Authentication: User registers or logs in via Firebase Authentication.
2. Scheduler Module: User inputs tasks, which are stored and dynamically prioritized in Firestore.
3. Doubt Solver: User inputs a query → Sent to GPT API → Response shown instantly.
4. Emotional Vent: User expresses emotions → GPT API responds with motivational text.
5. Focus Mode: Starts a countdown timer with music playback and logs session time.
6. Progress Tracker: Fetches and visualizes study data from Firestore.
7. Notifications: Motivational reminders pushed via FCM based on usage/stress detection patterns.

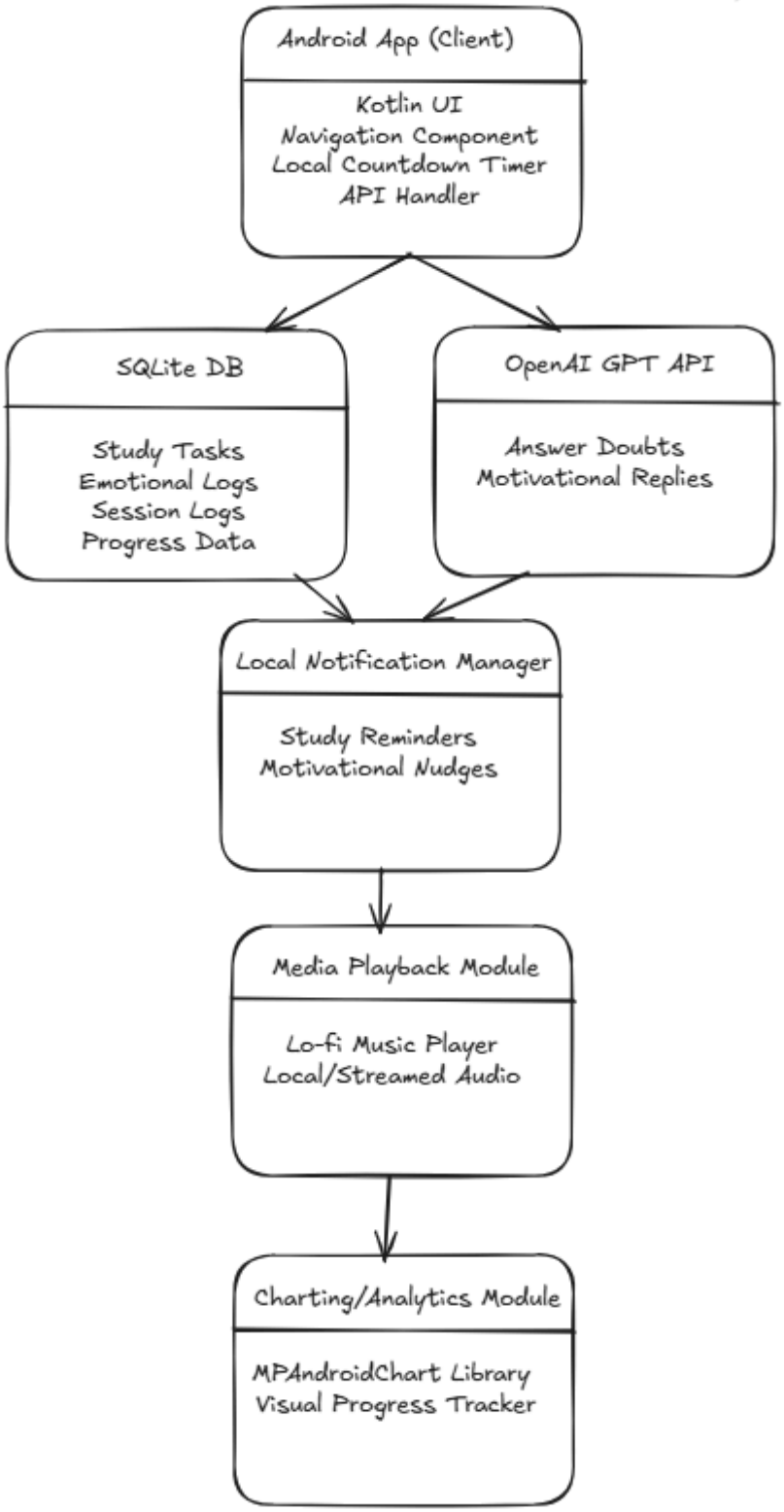
5. Security and Privacy

- All data transactions are secured with HTTPS.
- Firestore security rules ensure access control based on authenticated user sessions.
- Emotional vent data is stored with strict privacy, ensuring no unauthorized access.

6. Deployment and Maintenance

- The Android app is deployed via APK or Google Play Store.
- Firebase services are maintained via the Firebase Console.
- GPT API keys are securely stored using environment configurations or backend proxy (if needed).

Fig 3. Flow Chart



USER INTERFACE DESIGN AND NAVIGATION:

- **Dashboard**
  This is the central hub of StudyBuddy, providing quick access to all core features. Users can easily navigate to the Smart Scheduler, Doubt Solver, Emotional Vent Space, Focus Mode, and Easy Access Library through intuitive buttons. The layout is clean, minimal, and optimized for quick navigation.

- **Smart Scheduler**
  The Smart Scheduler screen helps users organize their daily tasks and deadlines. Students can add tasks using a floating action button, set due dates with a date picker, and view or manage existing tasks. This helps maintain productivity and reduces the chances of missing deadlines.

- **Focus Mode**
  This screen allows students to enter a distraction-free study session by setting a countdown timer. Lo-fi music suggestions or ambient background visuals help enhance concentration. It's a simple and effective way to stay on task during intense study sessions.

- **Emotional Vent Space**
  This space provides a safe outlet for students to express their feelings. Users can write their thoughts freely, and the app responds with calming, AI-generated affirmations or uplifting messages. It's designed to support mental wellness alongside academic productivity.

- **Easy Access Library**
  This section contains useful study materials, quick links to important academic resources, and personalized tips. It acts as a lightweight learning hub, allowing users to find what they need without leaving the app.

- **Doubt Solver**
  The Doubt Solver screen allows students to enter academic questions and receive instant responses powered by GPT. It's a quick-access tool to clarify doubts on the go — ideal for revision, last-minute preparation, or learning new concepts.

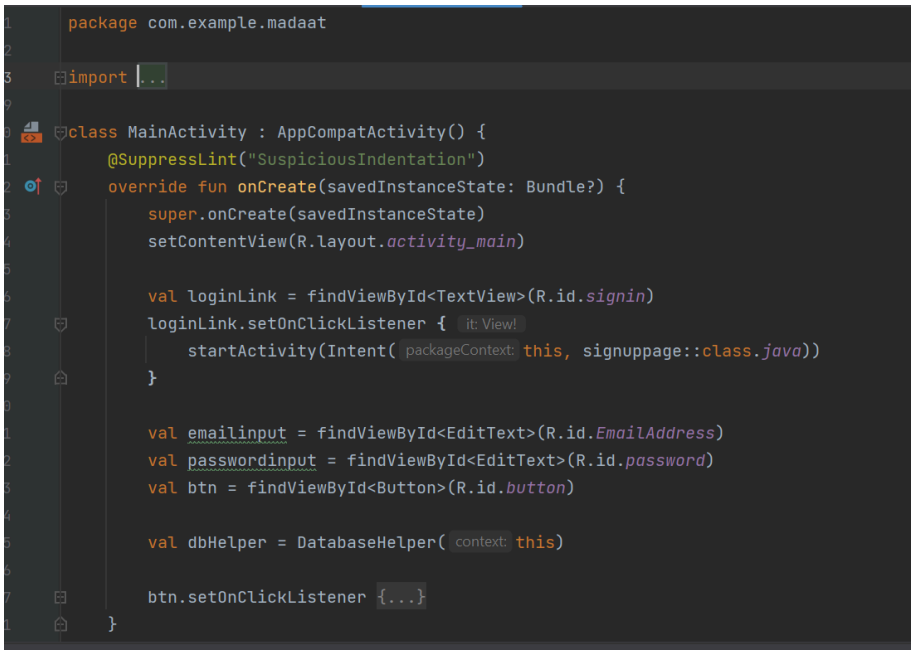
IMPLEMENTATION DETAILS:

The Student Study Sanity Assistant app is developed using Kotlin in Android Studio Dolphin, with a modular architecture designed for offline-first usability. Each core functionality is implemented using native Android components, local SQLite storage, and third-party APIs for intelligent responses.

1. User Authentication

- A simple local login/register mechanism is implemented using SharedPreferences for storing user credentials securely on the device.

- On successful login, the user is navigated to the main dashboard.



```kotlin
package com.example.madaat

import ...

class MainActivity : AppCompatActivity() {
    @SuppressLint("SuspiciousIndentation")
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val loginLink = findViewById<TextView>(R.id.signin)
        loginLink.setOnClickListener {  it: View!
            startActivity(Intent( packageContext: this, signuppage::class.java))
        }

        val emailinput = findViewById<EditText>(R.id.EmailAddress)
        val passwordinput = findViewById<EditText>(R.id.password)
        val btn = findViewById<Button>(R.id.button)

        val dbHelper = DatabaseHelper( context: this)

        btn.setOnClickListener {...}
    }
```

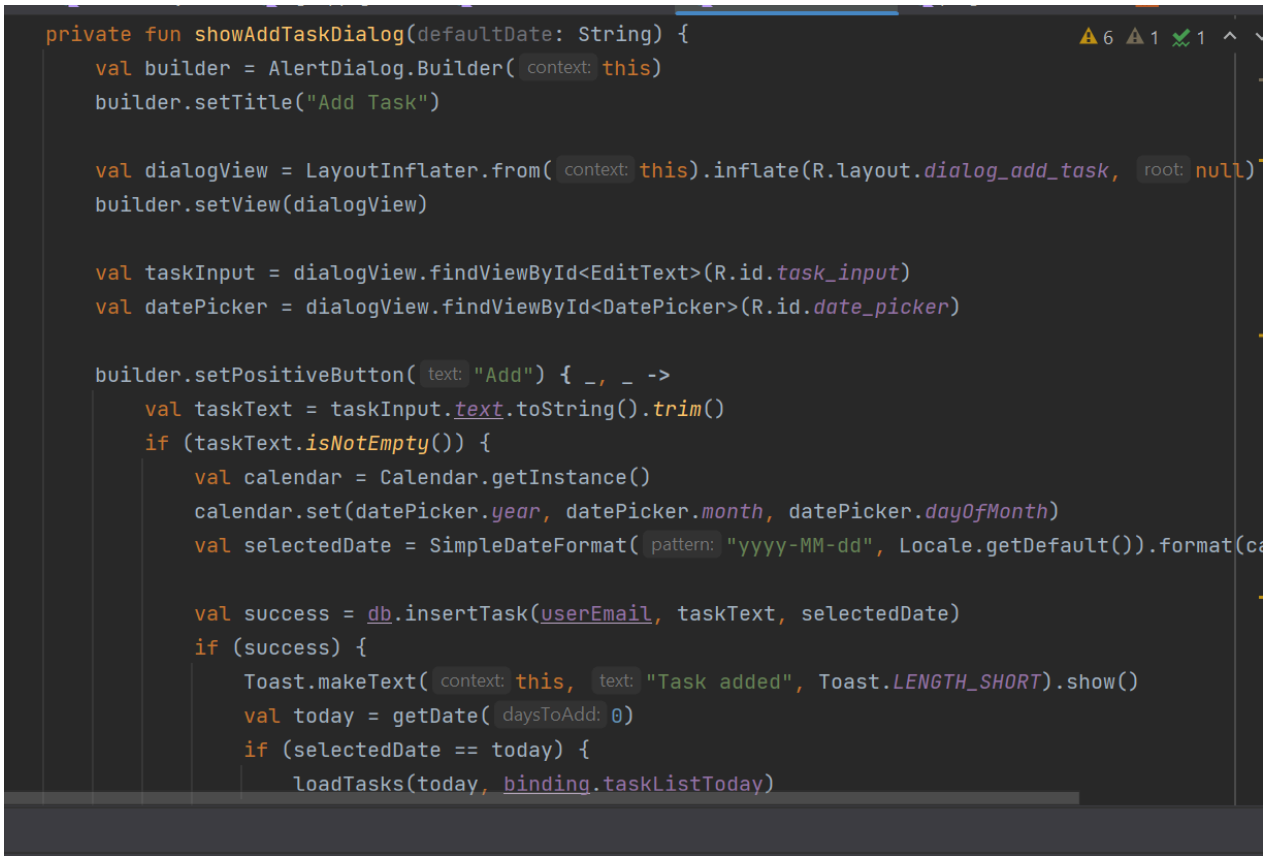
```
    btn.setOnClickListener {  it: View!
        val username = usernameField.text.toString().trim()
        val email = emailField.text.toString().trim()
        val password = passwordField.text.toString()
        val confirmPassword = confirmPasswordField.text.toString()

        if (validateSignUpForm(username, email, password, confirmPassword)) {
            val success = dbHelper.insertUser(username, email, password)
            if (success) {
                showToast("Signup successful")
                startActivity(Intent( packageContext: this, HomeDashboard::class.java))
                finish()
            } else {
                showToast("Sign Up failed: Email may already exist")
            }
        }
    }
    val allUsers = dbHelper.getAllUsers()
    for (user in allUsers) {
        Log.d( tag: "DB_USER", user)
    }

}
```

```
    private fun validateSignUpForm(
        username: String,
        email: String,
        password: String,
        confirmPassword: String
    ): Boolean {
        if (username.isEmpty()) {...}

        if (email.isEmpty() || !Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
            showToast("Enter a valid email")
            return false
        }

        if (password.length < 6) {
            showToast("Password must be at least 6 characters")
            return false
        }

        if (password != confirmPassword) {
            showToast("Passwords do not match")
            return false
        }
```

2. Smart Scheduler

- Users can add, update, and delete tasks using a clean form-based UI.
- Task attributes include: Title, Deadline, Priority, and Status.
- Tasks are stored in a SQLite database using a custom DAO (Data Access Object) class.

- The scheduler auto-sorts tasks based on Deadline and Priority using SQLite queries and Kotlin logic.



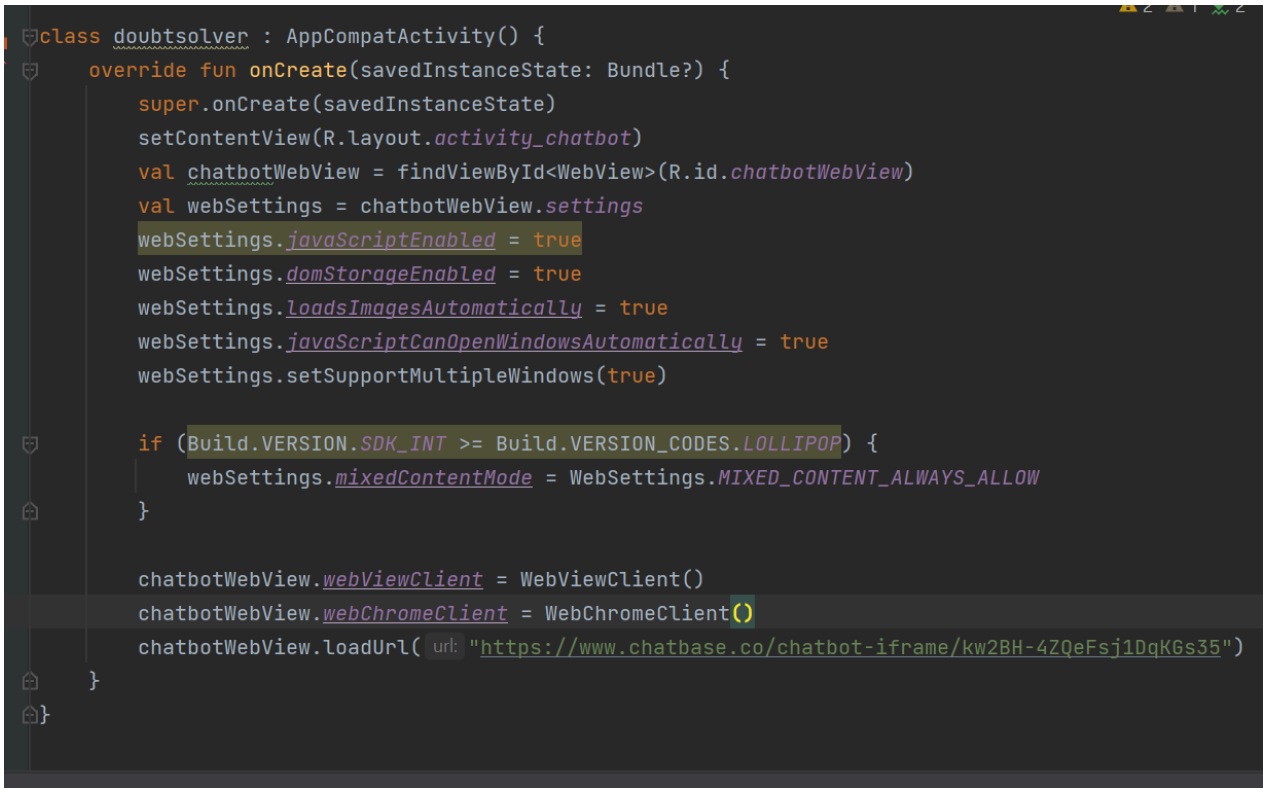
```
private fun showAddTaskDialog(defaultDate: String) {
    val builder = AlertDialog.Builder( context: this)
    builder.setTitle("Add Task")

    val dialogView = LayoutInflater.from( context: this).inflate(R.layout.dialog_add_task,  root: null)
    builder.setView(dialogView)

    val taskInput = dialogView.findViewById<EditText>(R.id.task_input)
    val datePicker = dialogView.findViewById<DatePicker>(R.id.date_picker)

    builder.setPositiveButton( text: "Add") { _, _ ->
        val taskText = taskInput.text.toString().trim()
        if (taskText.isNotEmpty()) {
            val calendar = Calendar.getInstance()
            calendar.set(datePicker.year, datePicker.month, datePicker.dayOfMonth)
            val selectedDate = SimpleDateFormat( pattern: "yyyy-MM-dd", Locale.getDefault()).format(ca

            val success = db.insertTask(userEmail, taskText, selectedDate)
            if (success) {
                Toast.makeText( context: this,  text: "Task added", Toast.LENGTH_SHORT).show()
                val today = getDate( daysToAdd: 0)
                if (selectedDate == today) {
                    loadTasks(today, binding.taskListToday)
```

```
private fun loadTasks(date: String, taskContainer: LinearLayout) {
    taskContainer.removeAllViews()
    val tasks = db.getTasks(userEmail, date)

    // If this is today's task list, update the task count text
    val today = getDate( daysToAdd: 0)
    Log.d( tag: "SmartScheduler",  msg: "Loading tasks for $date: ${tasks.size} found")
    if (date == today && taskContainer.id == binding.taskListToday.id) {
        binding.taskCountText.text = "You have ${tasks.size} tasks"
    }

    for (task in tasks) {
        val taskLayout = LinearLayout( context: this).apply {  this: LinearLayout
            orientation = LinearLayout.HORIZONTAL
            setPadding( left: 10,  top: 10,  right: 10,  bottom: 10)
            layoutParams = LinearLayout.LayoutParams(
                ViewGroup.LayoutParams.MATCH_PARENT,
                ViewGroup.LayoutParams.WRAP_CONTENT
            ).apply {  this: LinearLayout.LayoutParams
                setMargins( left: 0,  top: 8,  right: 0,  bottom: 8)
            }
        }
```

```
private lateinit var binding: ActivitySmartschedulerBinding
private lateinit var userEmail: String
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = ActivitySmartschedulerBinding.inflate(layoutInflater)
    setContentView(binding.root)

    db = DatabaseHelper( context: this)
    userEmail = intent.getStringExtra( name: "email") ?: "student@example.com"

    val currentDate = getDate( daysToAdd: 0)
    val tomorrowDate = getDate( daysToAdd: 1)

    binding.todayLabel.text = "Today, ${formatDisplayDate(currentDate)}"

    loadTasks(currentDate, binding.taskListToday)
    binding.viewScheduleButton.setOnClickListener {  it: View!
        loadTasks(tomorrowDate, binding.taskListTomorrow)
    }

    binding.addTaskButton.setOnClickListener {  it: View!
        showAddTaskDialog(currentDate)
    }
```

3. Instant Doubt Solver

- A query box allows users to input academic questions.
- On submission, the app sends a POST request to the OpenAI GPT API using Retrofit.
- The response is parsed and displayed within the same screen, using a RecyclerView for history.
- Error handling ensures that responses are gracefully managed in case of no network or invalid input.



```
class doubtsolver : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_chatbot)
        val chatbotWebView = findViewById<WebView>(R.id.chatbotWebView)
        val webSettings = chatbotWebView.settings
        webSettings.javaScriptEnabled = true
        webSettings.domStorageEnabled = true
        webSettings.loadsImagesAutomatically = true
        webSettings.javaScriptCanOpenWindowsAutomatically = true
        webSettings.setSupportMultipleWindows(true)

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.LOLLIPOP) {
            webSettings.mixedContentMode = WebSettings.MIXED_CONTENT_ALWAYS_ALLOW
        }

        chatbotWebView.webViewClient = WebViewClient()
        chatbotWebView.webChromeClient = WebChromeClient()
        chatbotWebView.loadUrl( url: "https://www.chatbase.co/chatbot-iframe/kw2BH-4ZQeFsj1DqKGs35")
    }
}
```

4. Emotional Vent Space

- Users can type in their thoughts or feelings in a text input area.
- The input is sent to GPT API, requesting an empathetic and motivational response.
- Replies are shown with a calming UI design to encourage emotional well-being.
- Entries are optionally saved in SQLite for users to revisit their emotional journey.

```
                // Reset animation properties
                inputField.text.clear()
                inputField.alpha = 1f
                inputField.scaleX = 1f
                inputField.scaleY = 1f

                // Optionally hide the EditText
                inputField.visibility = View.GONE

                // Show calming message
                calmingMessage.text = "Take a deep breath. You're doing great."
                calmingMessage.visibility = View.VISIBLE

                // Re-enable the button if needed
                letItOutButton.isEnabled = true
            }
        })
        .start()
    }
}
```

5. Focus Mode

- A custom countdown timer is implemented using Kotlin's `CountDownTimer` class.
- Upon activation, Lo-fi background music plays through:
    - Local `.mp3` assets bundled with the app, or
    - Streaming via MediaPlayer from a URL (if online).
- At the end of the timer, a popup confirms session completion, and the duration is logged in SQLite.

```
private fun pauseMusic() {
    mediaPlayer?.pause()
    nowPlayingText.text = "Music Paused"
}

private fun stopMusic() {
    mediaPlayer?.stop()
    mediaPlayer?.release()
    mediaPlayer = null
    nowPlayingText.text = "Now Playing: Lo-Fi Beats"
}

override fun onDestroy() {
    super.onDestroy()
    timer?.cancel()
    stopMusic()
}
```

```
private var timer: CountDownTimer? = null
private var timeLeftInMillis: Long = 25 * 60 * 1000 // default 25 mins
private var isRunning = false
private var mediaPlayer: MediaPlayer? = null
private var initialTimeInMillis: Long = timeLeftInMillis

override fun onCreate(savedInstanceState: Bundle?) {...}

private fun startTimer() {
    timer = object : CountDownTimer(timeLeftInMillis, countDownInterval: 1000) {
        override fun onTick(millisUntilFinished: Long) {
            timeLeftInMillis = millisUntilFinished
            updateTimerText()
        }

        override fun onFinish() {
            isRunning = false
            stopMusic()
            Toast.makeText( context: this@FocusMode, text: "Focus session complete!", Toast.LENGTH_SHORT
        }
    }.start()
    isRunning = true
}
```

```
private fun pauseTimer() {
    timer?.cancel()
    isRunning = false
}

private fun resetTimer() {
    timer?.cancel()
    timeLeftInMillis = initialTimeInMillis
    updateTimerText()
    isRunning = false
}

private fun updateTimerText() {
    val minutes = TimeUnit.MILLISECONDS.toMinutes(timeLeftInMillis)
    val seconds = TimeUnit.MILLISECONDS.toSeconds(timeLeftInMillis) % 60
    val formatted = String.format("%02d:%02d:%02d", minutes / 60, minutes % 60, seconds)
    timerText.text = formatted
}
```

6. Local Data Storage (SQLite)

- A custom SQLiteOpenHelper subclass manages the app's local database.
- Tables:
    - `users`: Stores user credentials.
    - `tasks`: Stores scheduler data.
    - `sessions`: Logs study time.
    - `vents`: Emotional logs.
- All database operations are abstracted through DAO-like helper methods.
- Ensures data persistence, offline access, and fast read/write.

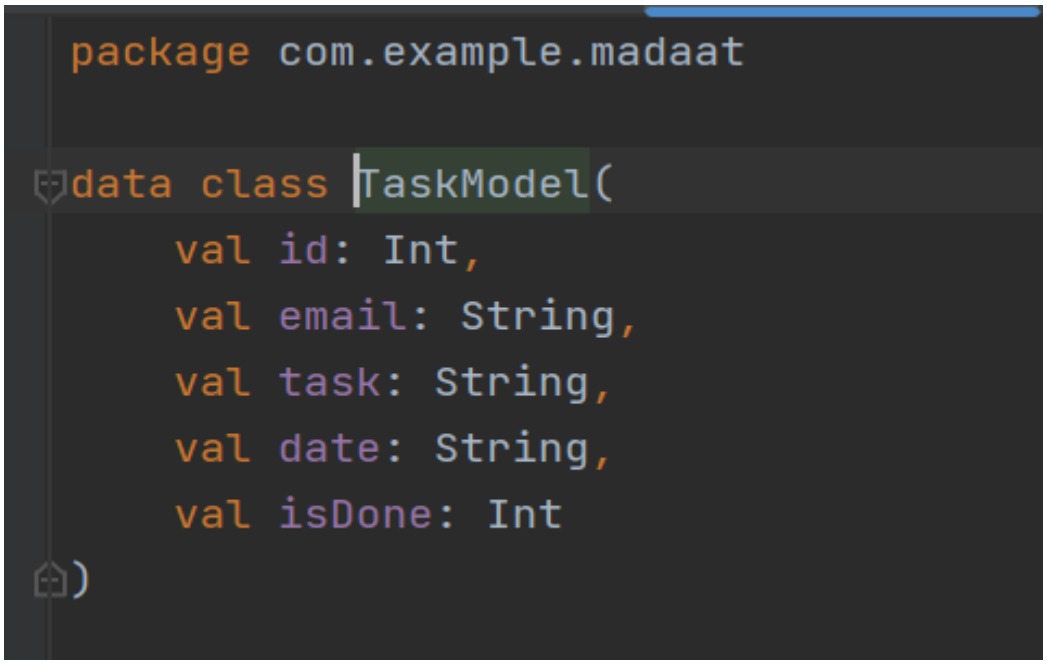
```
class DatabaseHelper(context: Context) : SQLiteOpenHelper(context, name: "UserDB", factory: null, version

    override fun onCreate(db: SQLiteDatabase) {
        val createTable = """
            CREATE TABLE users (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                username TEXT,
                email TEXT,
                password TEXT
            )
        """.trimIndent()
        db.execSQL(createTable)
        val createTasksTable = """
        CREATE TABLE tasks (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            email TEXT,  -- To link task to user
            task TEXT,
            date TEXT,
            isCompleted INTEGER DEFAULT 0
        )
    """.trimIndent()
```

```
    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
        db.execSQL( sql: "DROP TABLE IF EXISTS users")
        db.execSQL( sql: "DROP TABLE IF EXISTS tasks") // <-- ADD THIS LINE
        onCreate(db)
    }

    fun insertUser(username: String, email: String, password: String): Boolean {
        val db = this.writableDatabase
        val values = ContentValues().apply { this: ContentValues
            put("username", username)
            put("email", email)
            put("password", password)
        }
        val result = db.insert( table: "users", nullColumnHack: null, values)
        return result != -1L
    }
```

```kotlin
fun checkLogin(email: String, password: String): Boolean {
    val db = this.readableDatabase
    val cursor = db.rawQuery( sql: "SELECT * FROM users WHERE email = ? AND password = ?", arrayOf(ema
    val exists = cursor.count > 0
    cursor.close()
    return exists
}
fun getAllUsers(): List<String> {
    val db = this.readableDatabase
    val cursor = db.rawQuery( sql: "SELECT * FROM users", selectionArgs: null)
    val users = mutableListOf<String>()
    if (cursor.moveToFirst()) {
        do {
            val username = cursor.getString(cursor.getColumnIndexOrThrow("username"))
            val email = cursor.getString(cursor.getColumnIndexOrThrow("email"))
            users.add("Username: $username, Email: $email")
        } while (cursor.moveToNext())
    }
    cursor.close()
    return users
}
```

```kotlin
}
fun insertTask(email: String, task: String, date: String): Boolean {
    val db = this.writableDatabase
    val values = ContentValues().apply { this: ContentValues
        put("email", email)
        put("task", task)
        put("date", date)
        put("isCompleted", 0)
    }
    val result = db.insert( table: "tasks", nullColumnHack: null, values)
    return result != -1L
}
```

```kotlin
fun updateTaskDone(taskId: Int) {
    val db = this.writableDatabase
    val values = ContentValues().apply { this: ContentValues
        put("isCompleted", 1)
    }
    db.update( table: "tasks", values, whereClause: "id = ?", arrayOf(taskId.toString()))
}
fun deleteTask(taskId: Int) {
    val db = this.writableDatabase
    db.delete( table: "tasks", whereClause: "id = ?", arrayOf(taskId.toString()))
}


}
```



```
package com.example.madaat

data class TaskModel(
    val id: Int,
    val email: String,
    val task: String,
    val date: String,
    val isDone: Int
)
```

9. Navigation and UI Design

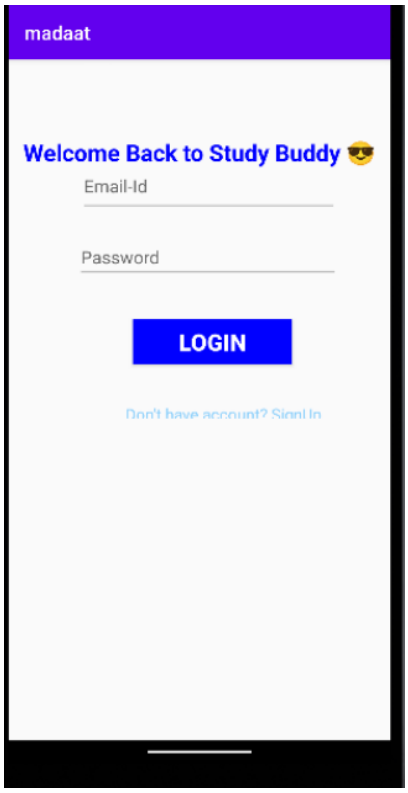
- Navigation is handled using Navigation Components with multiple fragments.
- Material Design principles are followed:
    - Consistent color palette.
    - Soothing UI for vent space.
    - Minimalist scheduler and progress dashboard.
- Each feature is isolated into its own module for better maintainability.

10. Error Handling and Optimization

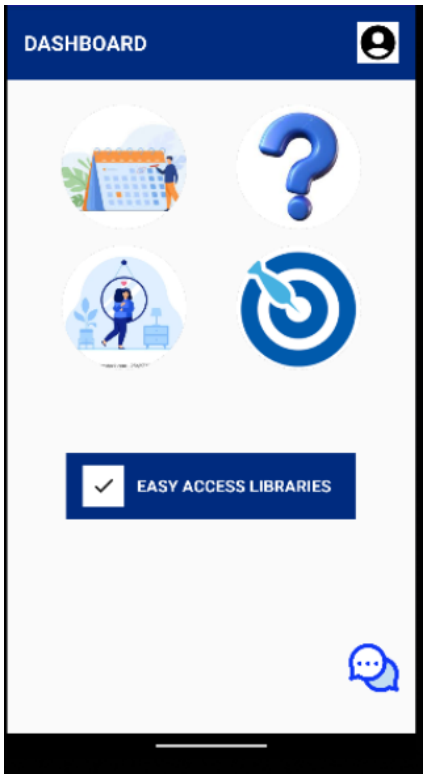
- All API calls are wrapped with try-catch blocks.
- Offline support ensures that users can continue using most features without the internet.
- Lightweight design ensures compatibility across Android versions and devices.

RESULTS:

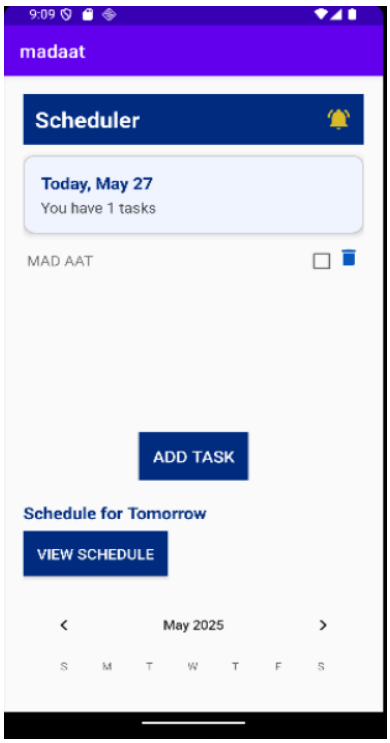
LOGIN/SIGNUP PAGE



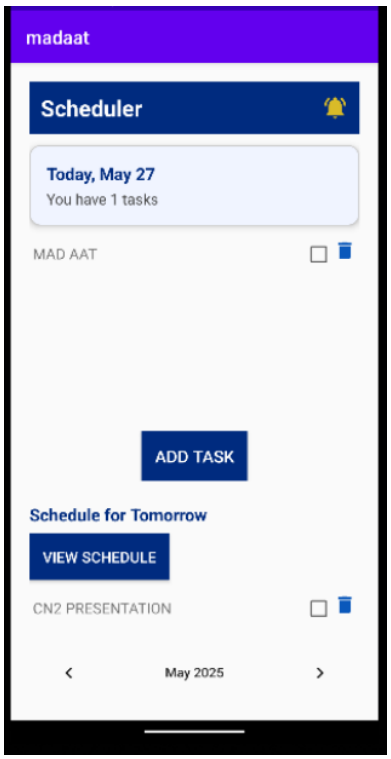
USER DASHBOARD



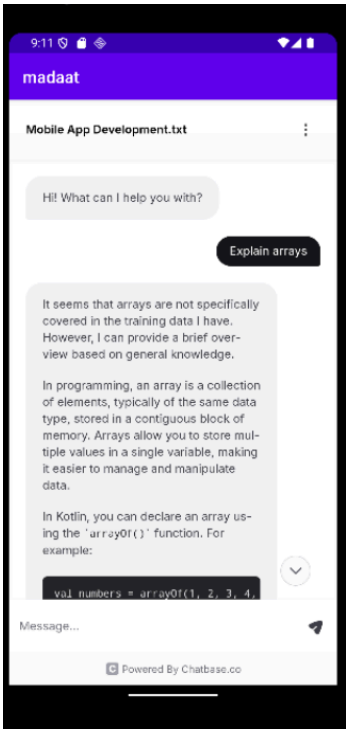
SCHEDULER PAGE



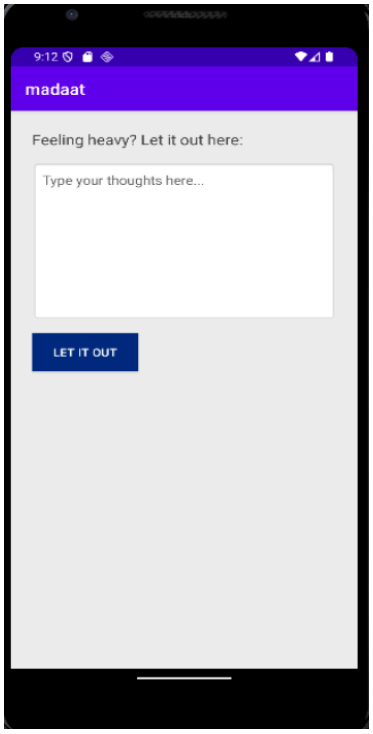
SCHEDULE TOMORROW PAGE



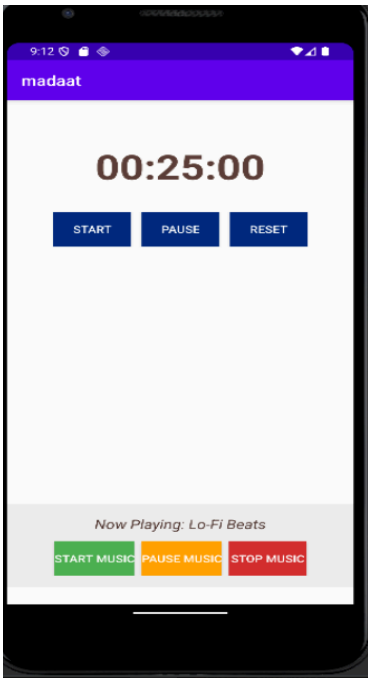
DOUBT SOLVER PAGE



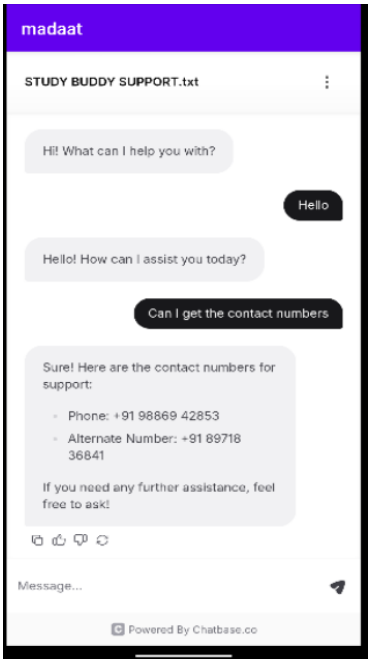
EMOTIONAL VENT PAGE



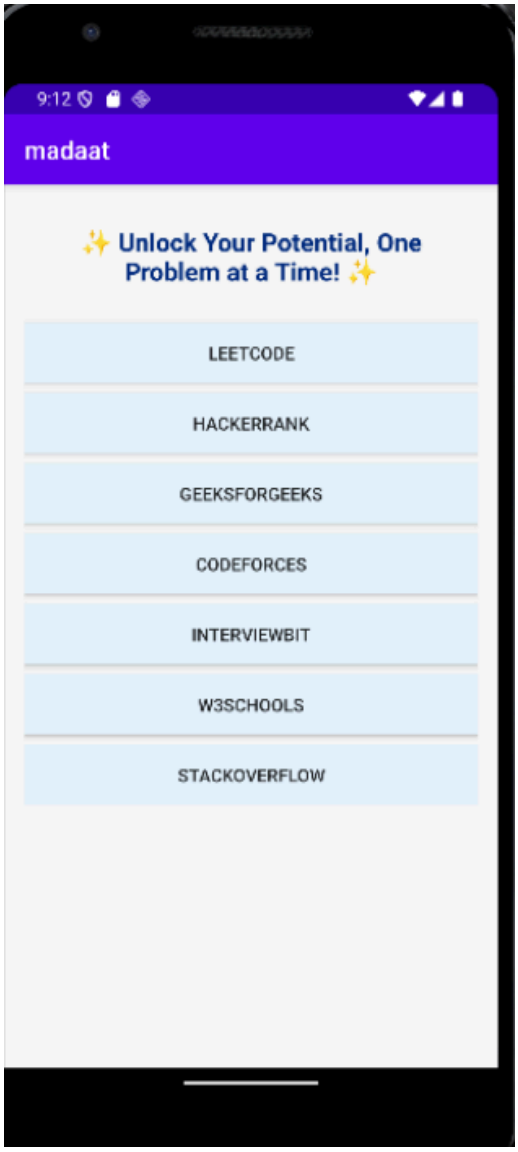
FOCUS MODE



HELP/SUPPORT PAGE



EASY ACCESS LIBRARY PAGE



CONCLUSION AND FUTURE ENHANCEMENT:

The Student Study Sanity Assistant successfully integrates essential tools for academic productivity and mental well-being into a single Android application. By combining a smart scheduler, AI-powered doubt solver, emotional venting space, focus timer, and progress tracker, the app addresses both the cognitive and emotional needs of students. The use of SQLite ensures secure and offline-first data storage, while OpenAI GPT API enables intelligent responses for doubts and emotional support.

The application's intuitive interface, responsive performance, and modular design make it a reliable companion for students striving to manage their time, focus effectively, and maintain mental balance during stressful academic phases.

To expand the capabilities and improve the user experience, several future enhancements can be implemented:

1. **Cloud Sync & Backup**
    - Integrate Firebase or Google Drive sync to allow users to back up their tasks and progress online and access them from multiple devices.
2. **Voice Input for Queries**
    - Enable users to ask doubts or vent emotions using speech-to-text, making the app more accessible and interactive.
3. **Gamification Elements**
    - Introduce badges, streaks, and reward points to encourage consistent usage and boost motivation.
4. **Mood Detection with Sentiment Analysis**
    - Analyze emotional inputs to detect the user's mood and provide more personalized motivational responses.
5. **Group Study Rooms (Collaborative Mode)**
    - Allow users to form groups, schedule joint sessions, and share resources or doubts in a peer-to-peer space.
6. **Dark Mode and Custom Themes**
    - Offer UI customization for better accessibility and user comfort, especially during

night study sessions.

7. **AI-Powered Study Plan Generator**
    - Automatically generate study plans based on syllabus inputs, deadlines, and available time using AI optimization techniques.
8. **Push Notification Personalization**
    - Use machine learning to send motivational nudges and reminders based on the user’s habits and usage patterns.
9. **Integration with Google Calendar**
    - Sync scheduled tasks and deadlines with the user's Google Calendar for external visibility and reminders.
10. **In-App Feedback & Analytics Dashboard**
    - Allow users to submit feedback and view detailed usage stats, helping both users and developers improve the app.

REFERENCES:

https://developer.android.com/reference

https://developers.google.com/android/guides/google-services-plugin

https://developer.android.com/develop/background-work/services/alarms/schedule

https://platform.openai.com/docs

https://developer.android.com/jetpack